DAIO Cross-Chain Governance Technical Reference — Agora Multichain Voting + LayerZero V2 "O" Standards

TL;DR

- **Use Agora's `lzVoteMain`/`lzVoteSide` lzRead pattern on the Polygon DAIO HubController and OAppOptionsType3-enforced OApps for slave executors on Base/Arbitrum/Ethereum/Optimism/Arc — but treat Agora's multichain module as a *design pattern* you re-implement, not a public dependency, because the lzVote contracts are not in any public voteagora repository as of May 2026.** Agora's underlying `AgoraGovernor` is a hardened, OZ-Governor-derived contract used in production by Uniswap, Optimism, ENS, Scroll, Derive, ether.fi, Cyber, B3, and Xai; the only publicly disclosed audit is Zach Obront's May 12, 2023 review of `voteagora/optimism-governor` at commit `35f441738bd7864bd37949a40842486bc0ac51b0`, and OpenZeppelin's Security Audits page states verbatim that "Our security researchers have identified 25+ issues in Lido's Dual Governance and 27+ issues in Agora's module-based voting, votable supply, and proposal validation" — but no formal OZ report is publicly hosted; no exploits or governance attacks against Agora-deployed governors have been publicly disclosed. Agora OpenZeppelin
- **All five DAIO EVM slaves can share the same LayerZero V2 EndpointV2 at `0x1a44076050125825900e736c501f859c50fE728c`**, but you must explicitly call `setConfig` with ≥2 required DVNs (LayerZero Labs + Nethermind, with Polyhedra + Google Cloud + Animoca-Blockdaemon as a 2-of-3 optional set) — Kelp DAO's $292M loss on April 18, 2026 was caused by a 1-of-1 DVN configuration; LayerZero later admitted ("We made a mistake by allowing our DVN to act as a 1/1 DVN for high-value transactions," CoinDesk, May 9, 2026), and the attack drained 116,500 rsETH from Kelp's LayerZero-powered bridge after attackers attributed to North Korea's Lazarus Group (TraderTraitor subunit) poisoned internal RPC nodes and DDoS-ed external ones to force the sole LayerZero Labs DVN to attest a forged cross-chain message. CoinDesk + 2
- **The recommended DAIO message flow is: vote on any chain → `lzVoteSide` reads ERC20Votes via lzRead DVNs → `lzVoteMain` aggregates on Polygon → AgoraGovernor `_castVote` consumes aggregated weight → TimelockController → OApp `_lzSend` to each slave → OAppPreCrimeSimulator-screened `_lzReceive` → slave TimelockController → execute.** Algorand constitutional layer reaches Polygon via Wormhole NTT (Algorand is not a LayerZero V2 chain).

Key Findings

1. Agora security & due diligence (confirmed)

- **Company**: Agora (`agora.xyz`, GitHub `voteagora`) is the onchain-governance company founded in 2022 by Yitong Zhang (ex-Coinbase designer), Charlie Feng (Clearco co-founder, CEO), and Kent Fenwick (ex-Clearco engineering executive). Per The Block (May 1, 2024), "Haun Ventures led the round, which had additional support from Seed Club, Coinbase Ventures, Sina Habibian, Balaji Srinivasan and others"; Fortune (May 1, 2024) reports a $5M seed announced Tuesday, May 1, 2024 with Consensys Ventures also participating. Code is MIT-licensed. The Block Fortune
- **Customers in production** (`agora.xyz` homepage): `vote.uniswapfoundation.org`, `gov.derive.xyz` (Lyra/Derive), `vote.ether.fi`, `gov.cyber.co`, `gov.xai.games`, `gov.b3.fun`, `gov.scroll.io`, Optimism Collective, ENS, Nouns.
- **Architecture**: `AgoraGovernor` is built on OpenZeppelin Governor; Agora is a founding member (with Tally, ScopeLift, and OpenZeppelin) of the OZ Governor Working Group. It adds: `ProposalTypesConfigurator` (per-type quorum/threshold), a `Middleware` contract that whitelists `VotingModule`s, partial delegation via `ERC20VotesPartialDelegationUpgradeable` ("Alligator"), and pluggable modules (Approval Voting, Optimistic, etc.). Agora
- **Audits found (public)**:
  - **Zach Obront — `optimism-governor` (May 12, 2023, commit `35f441738bd7864bd37949a40842486bc0ac51b0`).** Found a critical issue allowing a proposer to submit a malicious `VotingModule` that returns different `_formatExecuteParams()` than displayed; fix landed in commit `1152881afcb6272a29e80b0cb17914007a68cd27` as a module allowlist. Also flagged proposalId-uniqueness enforcement (fixed) and manager-editable proposal deadline. File: `voteagora/optimism-governor/audits/23-05-12_zachobront.md`. GitHub GitHub
  - **Zach Obront — Alligator (liquid-delegator)**, commits `38c41dd0c09b41f4597175cbd31e07e859fc3e2f` and re-review `c532d3e9e52e08c5a43d13cee3de8865fa143e00`. Findings on `ProxyV2.sol` EIP-1271 and the 36k `REFUND_BASE_GAS` gas-refund mechanism; fixed in PR #14. GitHub
  - The `voteagora/agora-governor/audits/` folder exists but its contents could not be enumerated in this research session; fetch it directly. OpenZeppelin's Security Audits page (openzeppelin.com/security-audits) states verbatim: "Our security researchers have identified 25+ issues in Lido's Dual Governance and 27+ issues in Agora's module-based voting, votable supply, and proposal validation" — meaning a substantive OZ engagement has occurred, though the formal report has not been located publicly.

- **No exploits, governance attacks, or vulnerability disclosures against Agora-deployed governors have been publicly disclosed.** Do not conflate Agora.xyz (voteagora) with "Agora Labs" / `agora.finance` (AUSD stablecoin issuer, audited by Cantina/Spearbit, Certora, Zellic, MoveBit in 2024) — they are different companies. Agora
- `lzVoteMain`/`lzVoteSide` **source code is not in any public voteagora or LayerZero-Labs repository as of May 2026.** Only the architecture and DVN-flexibility description in the Agora blog post (`agora.xyz/blogs/3-multichain-voting`) is public. Treat the design as a reference you re-implement and audit yourself.

2. Agora multichain voting architecture (deep dive)

From the Agora blog post (`agora.xyz/blogs/3-multichain-voting`), verbatim:

> "The read and aggregation logic is divided between **lzVoteMain**, deployed on the governor's chain, and **lzVoteSide**, deployed on each chain where the tokens exist. lzVoteSide reads voting power from ERC-20 contracts and sends the results to LayerZero's DVN, while lzVoteMain consolidates the data and provides a callback to the voting module." Agora

> "Through this governor module, voters are able to cast votes in a single transaction using voting power from any of the 100+ chains supported by LayerZero." Agora

> "Agora and apps using Agora for governance can currently leverage a diverse range of DVNs from entities like BCW, Nethermind, Animoca/Blockdaemon, Nocturnal, and AltLayer." Agora

Flow (lzRead pull model, not OApp push):

1. Voter calls `AgoraGovernor.castVote(proposalId, support)` on Polygon (hub).
2. The governor's voting module calls `lzVoteMain.requestVoteWeight(voter, snapshot)`.
3. `lzVoteMain` builds an `EVMCallRequestV1[]` (one entry per side chain) plus an `EVMCallComputeV1` (compute = `lzReduce`) using `ReadCodecV1.encode`.
4. Endpoint dispatches to the `ReadLib1002` channel (channelId `4294967295`). LayerZero
5. DVNs query `lzVoteSide.getVotingPower(voter, blockNumber)` on each chain at the snapshot block; each DVN response goes through `lzMap` for normalization.
6. `lzReduce` sums weights and returns one `uint256` total.
7. The total is delivered to `lzVoteMain._lzReceive`, which calls back into the AgoraGovernor module to finalize `_castVote(voter, proposalId, support, weight)`.

Why this is superior to OApp push aggregation (Wormhole MultiGov, OZ Cross-Chain Governor, Aragon Toucan): the voter pays one tx on one chain; DVN replay protection lives at the LayerZero ReadLib1002 channel (each channel has its own nonce sequence); no per-chain SpokeVoteAggregator deployment per voter action. The trade-off is that side chains must expose

ERC20Votes-compatible `getPastVotes(account, blockNumber)`, and side chains' blocks must be queryable by your DVN set at the snapshot. LayerZero

Contrast with Wormhole MultiGov (`HubGovernor`, `HubVotePool`, `HubMessageDispatcher`, `SpokeVoteAggregator`): MultiGov *pushes* aggregated tallies from each spoke via VAAs; voter must send a tx on each chain. Tally/ScopeLift's MultiGov uses OZ Governor with Flexible Voting on the hub. For DAIO, because the Algorand constitutional layer needs Wormhole NTT anyway, MultiGov is a viable alternative — but it forces multi-tx voting and a heavier deployment matrix. Wormhole

3. LayerZero V2 "O" standards reference

Reference paths and signatures are from `github.com/LayerZero-Labs/LayerZero-v2/packages/layerzero-v2/evm/oapp/contracts/`.

3a. `OApp` = `OAppSender` + `OAppReceiver` + `OAppCore`

Canonical skeleton (verbatim from the LayerZero V2 source):

```solidity
abstract contract OApp is OAppSender, OAppReceiver {
    constructor(address _endpoint, address _delegate) OAppCore(_endpoint, _delegate)
    function oAppVersion() public pure virtual returns (uint64 senderVersion, uint64
        senderVersion = SENDER_VERSION;
        receiverVersion = RECEIVER_VERSION;
    }
}
```

`OAppSender._lzSend(uint32 _dstEid, bytes memory _message, bytes memory _options, MessagingFee memory _fee, address _refundAddress)`.
`OAppReceiver._lzReceive(Origin calldata _origin, bytes32 _guid, bytes calldata _message, address _executor, bytes calldata _extraData)` — the source must equal `peers[_origin.srcEid]`. Replay protection is enforced by EndpointV2's `lazyInboundNonce`. LayerZero

DAIO HubController / Slave OApp (`MentisHubController.sol`):

```solidity
```

```solidity
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.22;

import { OApp, MessagingFee, Origin } from "@layerzerolabs/oapp-evm/contracts/oapp/O
import { OAppOptionsType3 } from "@layerzerolabs/oapp-evm/contracts/oapp/libs/OAppOp
import { TimelockController } from "@openzeppelin/contracts/governance/TimelockContr

/// @title MentisHubController — DAIO Hub on Polygon
contract MentisHubController is OApp, OAppOptionsType3 {
    uint16 public constant MSG_TYPE_PROPOSAL  = 1;
    uint16 public constant MSG_TYPE_EXECUTION = 2;
    uint16 public constant MSG_TYPE_VETO      = 3;

    TimelockController public immutable timelock;
    mapping(bytes32 => bool) public executedActions;

    event ProposalBroadcast(bytes32 indexed actionHash, uint32[] dstEids);

    constructor(address _endpoint, address _timelock)
        OApp(_endpoint, _timelock)
    { timelock = TimelockController(payable(_timelock)); }

    function broadcastExecution(
        bytes32 _wormholeVAAHash,
        uint32[] calldata _dstEids,
        bytes calldata _actionPayload,
        bytes calldata _extraOptions
    ) external payable {
        require(msg.sender == address(timelock), "Mentis: only timelock");
        require(!executedActions[_wormholeVAAHash], "Mentis: replay");
        executedActions[_wormholeVAAHash] = true;

        bytes memory msgPayload = abi.encode(MSG_TYPE_EXECUTION, _wormholeVAAHash, _
        uint256 totalFee;
        for (uint256 i = 0; i < _dstEids.length; i++) {
            bytes memory opts = combineOptions(_dstEids[i], MSG_TYPE_EXECUTION, _ext
            MessagingFee memory fee = _quote(_dstEids[i], msgPayload, opts, false);
            _lzSend(_dstEids[i], msgPayload, opts, fee, payable(msg.sender));
            totalFee += fee.nativeFee;
        }
        require(msg.value >= totalFee, "Mentis: insufficient fee");
        emit ProposalBroadcast(_wormholeVAAHash, _dstEids);
    }
```

```
    function _lzReceive(Origin calldata, bytes32, bytes calldata _message, address,
        internal override
    {
        (uint16 msgType, bytes32 actionHash) = abi.decode(_message, (uint16, bytes32
        require(msgType == MSG_TYPE_VETO, "Mentis: bad type");
        executedActions[actionHash] = false;
    }
}
```

3b. `OFT` — Omnichain Fungible Token (canonical for BKS and BKPY)

`OFT.sol` extends `OFTCore` + ERC20; transfers burn on source, mint on destination. `OFTAdapter.sol` locks an existing ERC20 in a single canonical chain (lockbox model). **Only one Adapter per token** across the entire mesh — multiple Adapters create unreconciled pools and the loss-of-funds race condition documented in `LayerZero-Labs/endpoint-v1-solidity-examples`. GitHub

solidity

```
// BankonSatoshi.sol — BKS as native OFT with Votes
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.22;
import { OFT } from "@layerzerolabs/oft-evm/contracts/OFT.sol";
import { ERC20Votes } from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Vote
import { ERC20Permit } from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Per
import { Nonces } from "@openzeppelin/contracts/utils/Nonces.sol";
import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract BankonSatoshi is OFT, ERC20Permit, ERC20Votes {
    constructor(address _lzEndpoint, address _delegate)
        OFT("BANKON SATOSHI", "BKS", _lzEndpoint, _delegate)
        ERC20Permit("BANKON SATOSHI")
    {}
    function _update(address f, address t, uint256 v) internal override(ERC20, ERC20
    function nonces(address o) public view override(ERC20Permit, Nonces) returns (ui
}
```

Notes:

- `_debit(msg.sender, amountLD, minAmountLD, dstEid)` / `_credit(toAddress, amountLD, srcEid)` are the override hooks.

- **Lossy quoting / shared decimals:** `OFTCore.decimalConversionRate = 10**(localDecimals - sharedDecimals)`. If BKS is 18-decimal locally but `sharedDecimals = 6`, dust under `10**12` rounds away in flight. Always set `minAmountLD` on send.
- For BKPY (governance token), prefer OFT over OFTAdapter to avoid lockbox single-point-of-failure on the canonical chain. Mint initial supply on one chain only (BasedOFT pattern from v1, manually replicated in V2 by minting in the constructor only when `block.chainid == ALGORAND_BRIDGE_CHAIN`). GitHub

3c. `ONFT721` — Omnichain ERC-721 (CONCLAVE cabinet, DELTAVERSE iDEBT)

Burn-and-mint by default; `ONFT721Adapter` lock-and-mint for pre-existing tokens. **Each chain must mint a disjoint tokenId range** (the example `ONFT_ARGS` in the LZ examples constrains nftId ranges per chain). For CONCLAVE (CEO + 7 Counsellors = 8 tokens total), this is trivial — pre-mint all 8 on one canonical chain and use Adapter pattern elsewhere. LayerZero GitHub

```solidity
import { ONFT721 } from "@layerzerolabs/onft-evm/contracts/onft721/ONFT721.sol";
contract ConclaveCabinet is ONFT721 {
    constructor(address _lzEndpoint, address _delegate)
        ONFT721("CONCLAVE Cabinet", "CONCLAVE", _lzEndpoint, _delegate) {}
}
```

For DELTAVERSE iDEBT (cross-chain inheritance), use `ONFT721Adapter` on the chain that mints debt positions (likely Polygon) and `ONFT721` clones on slaves. Override `_credit` to invoke an `IDebtInheritance.onArrival(uint256 tokenId, address beneficiary)` hook so the receiving chain re-attests the debt schedule.

3d. `ONFT1155` — AgenticPlace agent licenses

`ONFT1155` supports batch sends. Useful for AgenticPlace where one purchase grants {license:1, compute-credits:N, support-tier:K} atomically.

```solidity
import { ONFT1155 } from "@layerzerolabs/onft-evm/contracts/onft1155/ONFT1155.sol";
contract AgenticPlaceLicense is ONFT1155 {
    constructor(string memory _uri, address _lz, address _del) ONFT1155(_uri, _lz, _
}
```

3e. `OAppRead` — pull model (the core of DAIO multichain tally)

`OAppRead` extends `OAppSender` with the `ReadLib1002` channel (channelId `4294967295`). Implementers override `lzMap(bytes,bytes) → bytes` and `lzReduce(bytes,bytes[]) → bytes`, and build `EVMCallRequestV1[]` / `EVMCallComputeV1` payloads via `ReadCodecV1.encode`.

solidity

```solidity
// MasterTally.sol — DAIO Agora-style lzVoteMain re-implementation
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.22;
import { OAppRead } from "@layerzerolabs/oapp-evm/contracts/oapp/OAppRead.sol";
import { OAppOptionsType3 } from "@layerzerolabs/oapp-evm/contracts/oapp/libs/OAppOp
import { Origin, MessagingFee } from "@layerzerolabs/oapp-evm/contracts/oapp/OApp.so
import { IOAppMapper } from "@layerzerolabs/oapp-evm/contracts/oapp/interfaces/IOApp
import { IOAppReducer } from "@layerzerolabs/oapp-evm/contracts/oapp/interfaces/IOAp
import { ReadCodecV1, EVMCallRequestV1, EVMCallComputeV1 } from "@layerzerolabs/oapp

contract MasterTally is OAppRead, OAppOptionsType3, IOAppMapper, IOAppReducer {
    uint32 public constant READ_CHANNEL = 4294967295;
    struct SideChain { uint32 eid; address bkpyToken; uint16 confirmations; }
    SideChain[] public sideChains;
    mapping(uint256 => uint256) public aggregatedWeight;
    address public governor;

    event TallyRequested(uint256 indexed proposalId, address indexed voter, uint8 su
    event TallyResolved(uint256 indexed proposalId, uint256 totalWeight);

    constructor(address _endpoint, address _delegate) OAppRead(_endpoint, _delegate)

    function requestTally(uint256 proposalId, address voter, uint256 snapshotBlock,
        external payable returns (bytes32 guid)
    {
        require(msg.sender == governor, "Mentis: only governor");
        EVMCallRequestV1[] memory reqs = new EVMCallRequestV1[](sideChains.length);
        for (uint256 i = 0; i < sideChains.length; i++) {
            reqs[i] = EVMCallRequestV1({
                appRequestLabel: uint16(i),
                targetEid: sideChains[i].eid,
                isBlockNum: true,
                blockNumOrTimestamp: uint64(snapshotBlock),
                confirmations: sideChains[i].confirmations,
                to: sideChains[i].bkpyToken,
                callData: abi.encodeWithSignature("getPastVotes(address,uint256)", v
            });
        }
        EVMCallComputeV1 memory cmp = EVMCallComputeV1({
            computeSetting: 1, targetEid: 0, isBlockNum: false,
            blockNumOrTimestamp: 0, confirmations: 0, to: address(this)
        });
        bytes memory cmd = ReadCodecV1.encode(0, reqs, cmp);
```

```
        bytes memory opts = combineOptions(READ_CHANNEL, 1, abi.encode(proposalId, v
        MessagingFee memory fee = _quote(READ_CHANNEL, cmd, opts, false);
        return _lzSend(READ_CHANNEL, cmd, opts, fee, payable(msg.sender)).guid;
    }

    function lzMap(bytes calldata, bytes calldata _response) external pure returns (
    function lzReduce(bytes calldata, bytes[] calldata _responses) external pure ret
        uint256 total;
        for (uint256 i = 0; i < _responses.length; i++) total += abi.decode(_respons
        return abi.encode(total);
    }

    function _lzReceive(Origin calldata, bytes32, bytes calldata _msg, address, byte
        internal override
    {
        uint256 total = abi.decode(_msg, (uint256));
        (uint256 proposalId, address voter, uint8 support) = abi.decode(_extra, (uin
        aggregatedWeight[proposalId] += total;
        IGovernorCastback(governor).onCrossChainWeight(proposalId, voter, support, t
        emit TallyResolved(proposalId, total);
    }
}
```

Configuration: in `layerzero.config.ts` set `readLibrary` to `ReadLib1002` on Polygon, activate `ChannelId.READ_CHANNEL_1`, and specify `readConfig.ulnConfig.requiredDVNs` — minimum two (LayerZero Labs + Nethermind) and a 2-of-3 optional set (Polyhedra zkBridge + Google Cloud + Animoca-Blockdaemon). LayerZero

3f. `OAppPreCrimeSimulator` — pre-execution invariant checks

PreCrime forks the destination chain off-line, replays the inbound packet via `lzReceiveAndRevert`, and lets the application define invariants. If any invariant fails the DVN/Executor stack refuses to commit. `OAppPreCrimeSimulator.sol` exposes `_lzReceiveSimulate(Origin, bytes32, bytes, address, bytes)`, which routes the packet down through `OAppReceiver`. Medium GitHub

```solidity
```

```
import { OAppPreCrimeSimulator } from "@layerzerolabs/oapp-evm/contracts/precrime/OA

contract MentisSlavePreCrime is OAppPreCrimeSimulator, MentisSlaveExecutor {
    function _assertInvariants(bytes calldata _action) internal view {
        bytes4 sel = bytes4(_action[:4]);
        require(sel != IOwnable.transferOwnership.selector, "Mentis: ownership");
        require(sel != bytes4(0xff00ff00), "Mentis: selfdestruct guard");
        if (sel == ITreasury.withdraw.selector) {
            (, uint256 amt) = abi.decode(_action[4:], (address, uint256));
            require(amt + spentThisEpoch[currentEpoch()] <= TREASURY_CAP, "Mentis: c
        }
    }
}
```

The off-chain PreCrime relayer (run by LayerZero Labs and configurable per OApp) calls `lzReceiveAndRevert` over JSON-RPC; the simulated state is checked against the per-OApp invariant set declared in `getPreCrimePeers()` / `simulate()`.

3g. `OAppOptionsType3` — enforced execution options

`OAppOptionsType3.setEnforcedOptions(EnforcedOptionParam[] calldata)` forces a minimum gas/value bundle per (dstEid, msgType) that callers cannot under-fund. **Mandatory for DAIO** to prevent griefing attacks where someone sends an execution with insufficient gas, stalling `_lzReceive`. GitHub

solidity

```
EnforcedOptionParam[] memory opts = new EnforcedOptionParam[](1);
opts[0] = EnforcedOptionParam({
    eid: 30184, // Base
    msgType: MSG_TYPE_EXECUTION,
    options: OptionsBuilder.newOptions().addExecutorLzReceiveOption(500_000, 0)
});
hub.setEnforcedOptions(opts);
```

3h. `lzCompose` — horizontal composability

`EndpointV2.sendCompose(toAddress, _guid, index, composeMsg)` is called inside `_lzReceive`; the off-chain executor later calls `lzCompose` on the target contract (which implements `ILayerZeroComposer`) in a separate tx. This decouples receive logic from downstream call logic — critical when the downstream call could revert and you want the primary receive to still commit.

solidity

```solidity
import { ILayerZeroComposer } from "@layerzerolabs/lz-evm-protocol-v2/contracts/inte
import { OFTComposeMsgCodec } from "@layerzerolabs/oft-evm/contracts/libs/OFTCompose

contract BankonComposeOrder is ILayerZeroComposer {
    address public immutable endpoint;
    address public immutable bksOFT;
    AgenticPlaceLicense public immutable license;
    function lzCompose(address _from, bytes32, bytes calldata _msg, address, bytes c
        require(msg.sender == endpoint, "!endpoint");
        require(_from == bksOFT, "!BKS");
        uint256 amtLD = OFTComposeMsgCodec.amountLD(_msg);
        bytes memory app = OFTComposeMsgCodec.composeMsg(_msg);
        (address buyer, uint256 tokenId, uint256 qty) = abi.decode(app, (address, ui
        require(amtLD >= license.priceOf(tokenId) * qty, "underpaid");
        license.mint(buyer, tokenId, qty, "");
    }
}
```

4. DAIO Agora-pattern reference implementation

`AgoraDAIOGovernor.sol`

Inherits `AgoraGovernor` and overrides `_castVote` to dispatch a `MasterTally.requestTally` instead of consuming local-only weight, then resolves the vote in the `onCrossChainWeight` callback. Uses `Middleware` to register a `BonaFideVotesAdapter` voting module that gates by an EAS-style attestation registry.

solidity

```
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.22;
import { AgoraGovernor } from "voteagora/agora-governor/src/AgoraGovernor.sol";

contract AgoraDAIOGovernor is AgoraGovernor {
    MasterTally       public immutable tally;
    BonaFideAttestor  public immutable bonafide;
    ConvictionVoting  public immutable conviction;
    mapping(uint256 => mapping(address => uint8)) public pendingSupport;

    constructor(/* args */) AgoraGovernor(/* args */) {}

    function _castVote(uint256 pid, address voter, uint8 support, string memory, byt
        internal virtual override returns (uint256)
    {
        require(bonafide.isAttested(voter), "DAIO: !BONAFIDE");
        uint256 snap = proposalSnapshot(pid);
        pendingSupport[pid][voter] = support + 1;
        tally.requestTally{value: msg.value}(pid, voter, snap, support);
        return 0;
    }

    function onCrossChainWeight(uint256 pid, address voter, uint8 support, uint256 b
        require(msg.sender == address(tally), "DAIO: !tally");
        uint256 boosted = conviction.boost(voter, baseWeight);
        super._countVote(pid, voter, support, boosted, "");
    }
}
```

`BonaFideVotesAdapter.sol` — attestation gating

solidity

```
contract BonaFideAttestor {
    IEAS public immutable eas;
    bytes32 public immutable schemaUid;
    function isAttested(address who) external view returns (bool) {
        Attestation memory a = eas.getAttestation(_uidFor(who, schemaUid));
        return a.attester != address(0) && a.revocationTime == 0 &&
               (a.expirationTime == 0 || a.expirationTime > block.timestamp);
    }
}
```

`ConvictionVoting.sol` — duration-weighted weight

solidity

```
contract ConvictionVoting {
    mapping(address => uint64) public lockStart;
    function boost(address voter, uint256 base) external view returns (uint256) {
        uint256 dur = block.timestamp - lockStart[voter];
        uint256 mult = 1e18 + (dur * 1e18) / 180 days;
        if (mult > 2e18) mult = 2e18;
        return (base * mult) / 1e18;
    }
}
```

`AgoraVotingChain.sol` — `lzVoteSide` on each slave

This is just BKPY itself (§3b) — `getPastVotes(address, uint256)` is the queried function. No additional contract needed unless you want to add a per-chain vote-weight multiplier (e.g., Arc gets 1.5× to encourage participation).

5. Integration with existing DAIO architecture

```
                                                                Execute
(treasury, contract calls, OFT/ONFT mints)
```

Veto path: Security Council slave → `_lzSend(MSG_TYPE_VETO, actionHash)` back to Hub → `executedActions[h] = false`.

6. Production deployment addresses (verified)

Chain	EID	EndpointV2
Ethereum	30101	`0x1a4407605012582590 0e736c501f859c50fE728c`
Polygon	30109	`0x1a4407605012582590 0e736c501f859c50fE728c`
Arbitrum One	30110	`0x1a4407605012582590 0e736c501f859c50fE728c`
Optimism	30111	`0x1a4407605012582590 0e736c501f859c50fE728c`
Base	30184	`0x1a4407605012582590 0e736c501f859c50fE728c`

Ethereum verified library addresses: SendUln302 = `0xbB2Ea70C9E858123480642Cf96acbcCE1372dCe1`; ReceiveUln302 = `0xc02Ab410f0734EFa3F14628780e6e695156024C2`; Executor = `0x173272739Bd7Aa6e4e214714048a9fE699453059`; LayerZero Labs DVN = `0x589dEDbD617e0CBcB916A9223F4d1300c294236b` (this was the single-DVN in the Kelp DAO compromise — *which is exactly why DAIO must use ≥2 required DVNs*). GitHub GitHub

Optimism ReceiveUln302: `0x3c4962Ff6258dCfCAfd23a814237b7d6Eb712063`.
Arbitrum One ReceiveUln302: `0x7B9E184e07a6EE1aC23EAe0fe8D6Be2f663f05e6`; Arbitrum Executor: `0xe93685f3bBA03016F02bD1828BaDD6195988D950`.

For complete per-chain addresses (SendUln302, ReceiveUln302, Executor, all DVN providers) on Polygon, Base, and the missing Optimism/Arbitrum SendUln/DVN entries, pull the canonical table from `github.com/LayerZero-Labs/lz-address-book` (`src/generated/LZAddresses.sol`). That repo regenerates every 6 hours from official LayerZero metadata; do not rely on snapshots. The "Arc" chain is not yet a documented LayerZero V2 deployment as of May 2026; verify EID availability before designing for Arc. github

7. Foundry test patterns

Use `TestHelperOz5` from `@layerzerolabs/test-devtools-evm-foundry`:

```
solidity
```

```
contract DAIOMultichainTest is TestHelperOz5 {
    using OptionsBuilder for bytes;
    uint32 constant POLY_EID = 1;
    uint32 constant BASE_EID = 2;
    MentisHubController hub;
    MentisSlaveExecutor slave;

    function setUp() public override {
        super.setUp();
        setUpEndpoints(2, LibraryType.UltraLightNode);
        address[] memory oapps = setupOApps(type(MentisHubController).creationCode,
        hub   = MentisHubController(payable(oapps[0]));
        slave = MentisSlaveExecutor(payable(oapps[1]));
    }
    function test_BroadcastFlow() public {
        uint32[] memory dst = new uint32[](1); dst[0] = BASE_EID;
        bytes memory opts = OptionsBuilder.newOptions().addExecutorLzReceiveOption(5
        hub.broadcastExecution{value: 0.01 ether}(keccak256("vaa"), dst, abi.encode(
        verifyPackets(BASE_EID, addressToBytes32(address(slave)));
    }
}
```

Fork tests: use `lz-address-book` `LZAddressContext` helper with `setChainByEid(30109)` for Polygon and `makePersistent(vm)` for multi-fork tests. PreCrime simulation uses `lzReceiveAndRevert` against the forked state.

8. Production deployment checklist

1. **Pre-deploy**: Slither + Mythril on all contracts; Foundry fork tests on Polygon/Base/Arb/ETH/Op; PreCrime invariant tests; DVN config diff vs. `lz-address-book`.
2. **Deploy** via CREATE3 deterministic across chains (LayerZero devtools provides `@layerzerolabs/create3-factory` helpers).
3. **`setPeer`** on every (src,dst) pathway. Use `npx hardhat lz:oapp:wire --oapp-config layerzero.config.ts`.
4. **`setConfig`** with `UlnConfig` containing `requiredDVNCount=2` (LayerZero Labs + Nethermind) and `optionalDVNCount=3, optionalDVNThreshold=2` (Polyhedra + Google Cloud + Animoca-Blockdaemon). Set `confirmations` per-pathway to ≥15 on Polygon and ≥10 on Arbitrum (verify against `lz-address-book` recommended defaults).
5. **`setEnforcedOptions`** with 500k gas for execution messages, 200k for veto.

6. **Transfer ownership** of every OApp to the chain's TimelockController; transfer Timelock admin to a Safe multisig (2-of-3 founder + 2 Counsellors).
7. **Monitoring**: LayerZero Scan webhooks → PagerDuty; Tenderly War-Room on each hub/slave; OpenZeppelin Defender Sentinel on `ProposalBroadcast`/`TallyResolved`/`executedActions` events.
8. **Incident response**: `endpoint.skip(srcEid, sender, nonce)` to discard a stuck packet; `setConfig` migration to swap DVNs if a provider is compromised; Security Council veto path; emergency `pause()` on the OApp.

9. Alternatives if Agora's lzRead path is unworkable

Option	Pros	Cons
Wormhole MultiGov (Tally/ScopeLift)	Production with Wormhole DAO; Wormhole hub-and-spoke matches existing DAIO Wormhole leg; **audited by Zellic, January 14–31, 2025, for the Wormhole Foundation** Zellic **(reports.zellic.io/publications/multigov)**	Requires per-chain SpokeVoteAggregator deployment; voter must tx on each chain
OZ Cross-Chain Governor	Same OZ Governor base as Agora; simpler	No native multichain aggregation; you still build the bridge
Aragon OSx Toucan Voting	Gasless on ZKsync paymaster; Aragon's Blog LayerZero V2 already integrated; excellent permission system	Tightly coupled to Aragon OSx framework; less flexibility for BONAFIDE/Conviction modules
Snapshot off-chain + on-chain execution	Cheapest UX	Off-chain trust; doesn't satisfy DAIO's "all on-chain" constitutional requirement
Aave a.DI (BUSL-1.1)	Battle-tested with two-bridge consensus; multiple audits (MixBytes Polygon, Certora formal verification, ChainSecurity, Oxorio)	Licensed BUSL-1.1 (not Apache-2.0); designed for execution not vote aggregation

Recommendation: build the Agora-pattern lzRead `MasterTally` yourself (since the lzVote contracts are not public), keep Wormhole MultiGov (Zellic-audited) as the proven fallback if

your audit team rejects an unaudited bespoke lzRead implementation, and reserve Aragon OSx only if you want gasless voting subsidized via ZKsync paymaster.

Recommendations (staged)

Phase 0 (now → +30 days): Fork `voteagora/agora-governor` and `voteagora/optimism-governor`; deploy on Polygon Amoy testnet with a single ERC20Votes BKPY-test and no cross-chain weight. Verify ProposalTypesConfigurator and Middleware behavior end-to-end against an Apache-2.0 relicensed fork (Agora's code is MIT; MIT-to-Apache-2.0 is compatible). **Threshold to advance:** full proposal lifecycle (propose → vote → queue → execute) green on testnet.

Phase 1 (+30 → +90 days): Build `MasterTally` lzRead implementation (your own lzVoteMain) and BKPY OFT+ERC20Votes on Polygon + Base + Arbitrum testnets. Set up two required DVNs + 2-of-3 optional. Foundry coverage ⩾95%. **Threshold to advance:** TestHelperOz5 tests pass and a Polygon-Amoy → Base-Sepolia → Polygon-Amoy round-trip vote tallies correctly under 90 seconds.

Phase 2 (+90 → +150 days): Engage OpenZeppelin and Spearbit/Cantina for parallel audits. Specific scope: `AgoraDAIOGovernor`, `MasterTally` (lzRead), `MentisHubController`, `MentisSlaveExecutor`, `OAppPreCrimeSimulator` invariants, `BonaFideVotesAdapter`, `ConvictionVoting`, plus all OFT/ONFT contracts. **Threshold to advance:** 0 critical, 0 high, ⩽3 medium, all fixed and re-reviewed.

Phase 3 (+150 → +210 days): Mainnet deployment via CREATE3 to Polygon, Base, Arbitrum, Ethereum, Optimism (Arc only when LayerZero V2 confirms an EID). Algorand-side Wormhole NTT integration tested with replay-attack and VAA-malleability suites. **Threshold to go live:** 30-day bug bounty live on Immunefi at ⩾$500k max payout, Tenderly + Defender alerts green, multisig training complete.

Threshold to NOT proceed with lzRead and instead fall back to Wormhole MultiGov: if audit firms refuse to sign off on the bespoke `MasterTally` (because the upstream `lzVoteMain` is unaudited and private), or if LayerZero V2 lzRead has not yet been audited end-to-end by a top-tier firm at the time of your audit kickoff. MultiGov has been audited by Zellic (January 14–31, 2025, for Wormhole Foundation) and is in production with Wormhole's own DAO. Zellic

---

Caveats

- **The Agora `lzVoteMain`/`lzVoteSide` contracts are not publicly available** in any voteagora or LayerZero-Labs repo as of May 2026. Treat the Agora blog post as a design

specification, not as importable code. The reference implementation in §4 above is original and unaudited.

- **No formal Agora multichain audit report has been located publicly.** OpenZeppelin's Security Audits page states "27+ issues" identified in Agora's module-based voting, but no report is on `blog.openzeppelin.com` or `docs.agora.xyz/audits`. Confirm with Agora directly before adopting their multichain stack in production.
- **LayerZero V2 lzRead is newer than core OFT/OApp** and its audit history is thinner. Multi-DVN configuration is the primary defense; 1-of-1 DVN is the precise misconfiguration that lost Kelp DAO $292M on April 18, 2026 — the exploit drained 116,500 rsETH (~18% of rsETH's 630,000-token circulating supply) when North Korea–attributed attackers (Lazarus Group's TraderTraitor subunit) poisoned internal RPC nodes and DDoS-ed external ones to force the sole LayerZero Labs DVN to attest a forged message at 17:35 UTC, making it the largest DeFi exploit of 2026 (CoinDesk, April 19, 2026); LayerZero subsequently admitted, "We made a mistake by allowing our DVN to act as a 1/1 DVN for high-value transactions" (CoinDesk, May 9, 2026). This was an application-layer DVN config issue, not a LayerZero protocol failure — but it is the single strongest cautionary signal for any team using lzRead. CoinDesk + 5
- **Algorand is not a LayerZero V2 chain.** The Algorand → Polygon leg is Wormhole NTT; LayerZero only carries the EVM-to-EVM legs.
- **Arc** (referenced in the user's chain list) is not a confirmed LayerZero V2 deployment in publicly available LayerZero docs as of May 2026. Verify EID/endpoint availability with LayerZero before designing for Arc.
- **"Agora" name collision**: Agora.xyz (governance) ≠ Agora Labs / agora.finance (AUSD stablecoin) ≠ agora.vote (Swiss election lab) ≠ Agora.io (RTC/streaming SDK). Audit references in §1 are strictly Agora.xyz / voteagora.
- **Per-chain SendUln302, ReceiveUln302, Executor, and DVN addresses for Polygon, Base, and the missing Optimism/Arbitrum entries must be re-pulled from `github.com/LayerZero-Labs/lz-address-book` at deployment time** — those tables regenerate every 6 hours and snapshots in this document can drift.
- Production deployment of governance contracts that handle treasury value should always include a bug bounty (Immunefi recommended ⩾$500k cap for ⩾$5M TVL), a pause/guardian role with a 7-of-15 council, and a forced cool-down between proposal types — none of which are LayerZero or Agora specific but all of which Agora's `ProposalTypesConfigurator` makes ergonomic.